1. Problem Statement

In a warehouse environment, keeping track of inventory manually often leads to discrepancies between recorded stock and physical stock due to miscounts, damage, or unrecorded movements. The objective is to automate the reconciliation process by calculating the **expected closing stock** based on opening stock, received goods, and dispatched goods, and comparing it against the **actual physical stock** to identify shortages or surpluses.

2. Objectives

- To compute the expected closing stock using the inventory formula: $\text{Expected} = \text{Opening} + \text{Received} - \text{Dispatched}$.
- To compare the expected stock with the actual physical count.
- To generate an automated stock report categorizing each product's status as **Matched**, **Shortage**, or **Surplus**.

3. Assumptions

- The user inputs valid integer values for quantities and valid text strings for product names.
- "Opening stock," "Received quantity," and "Dispatched quantity" are non-negative numbers.
- The physical inventory count (actual) is performed accurately at the end of the reporting cycle.

- ## 4. Algorithm

1. **Start**
2.
3. Read the number of products (n) from the user.
4.
5. Repeat the following steps for each product i from 1 to n:
6.
   - Read product name.
   -
   - Read opening stock, received quantity, and dispatched quantity.
   -
   - Read actual physical closing stock.
   -
   - Calculate $\text{expected} = \text{opening} + \text{received} - \text{dispatched}$.
   -
   - Calculate $\text{difference} = \text{actual} - \text{expected}$.
   -
   - Store the tuple (name, expected, actual, difference) in the products list.
   -
7. Print the header --- STOCK REPORT ---.
8.
9. For each product stored in the list:

10.
    - Print its name, expected stock, and actual stock.
    -
    - If difference == 0, print status as **Matched**.
    -
    - If difference < 0, print status as **Shortage** equal to abs(difference).
    -
    - If difference > 0, print status as **Surplus** equal to difference.
    -
11. **End**
12. ## 5. Flowchart (Text-Based)

6. Pseudocode

7. Python Source Code

(Provided in your source code)

```
    int input "Number of products: "
for   in range
            input "Product name: "
              int input "Opening stock: "
              int input "Received quantity: "
                 int input "Dispatched quantity: "
             int input "Actual closing stock: "
```

```
      "\n--- STOCK REPORT ---" for                                    in

                  "| Expected:"            "| Actual:"
  if                  0
            "Status: Matched"
  elif                0
            "Status: Shortage of"  abs
  else
            "Status: Surplus of"
```

8. Test Cases

Test Case ID	Inputs (n, name, opening, received, dispatched, actual)	Expected Output / Status

Test Case ID	Inputs (n, name, opening, received, dispatched, actual)	Expected Output / Status
TC-01 (Matched)	1, "Pen", 100, 50, 30, 120	Expected: 120, Actual: 120 $\rightarrow$ **Status: Matched**
TC-02 (Shortage)	1, "Notebook", 200, 20, 50, 160	Expected: 170, Actual: 160 $\rightarrow$ **Status: Shortage of 10**
TC-03 (Surplus)	1, "Marker", 50, 10, 5, 60	Expected: 55, Actual: 60 $\rightarrow$ **Status: Surplus of 5**

9. Output Screenshots (Simulation)

10. Conclusion

This Python program successfully automates warehouse inventory reconciliation. By comparing the mathematically expected stock against the physical stock, it helps warehouse managers quickly detect discrepancies (shortages or surpluses), improving stock accuracy and accountability.